real-mb-fidelity___fidelity-chain-f

An AgentMPI run analysis

Abstract

This is the deep cell of the reduction-fidelity experiment: eight ranks, four uniquely identified facts each, folded to rank 0 by a model-executed merge arranged as a serial chain of depth 7 — the maximum possible at $p = 8$, and the configuration the hypothesis under test predicted would lose the most. It loses nothing. No rank was erased. Retention is 1.000, all 32 identifiers reach the root, none is fabricated, and this is not merely the aggregate: I recovered all seven intermediate accumulators from this run's `spool/` directory under `runs/real-mb-fidelity/` and each carries exactly four identifiers per rank folded into it — 8, 12, 16, 20, 24, 28, 32 — growing by four at every hop with nothing dropped along the way. The shallow sibling `-binomial-f` (depth 3) scores the same 1.000, as does `-flat-f`. Seven successive lossy summarisations cost exactly as much fidelity as three, namely none, so the tree-shape hypothesis finds no support here. The reason it could not have been tested is twofold: the merges peaked at 622 output tokens against a nominal 900-token budget, and the contract recorded in the fabric carries `max_tokens: null`, so the budget was a sentence in a prompt rather than a constraint. Depth-dependent loss does not appear in the enforced, incompressible sibling campaign either — `inc-mb-fidelity-inc` scores 0.385 identically at depths 2, 3, 7 and 7 — so what governs retention is the enforced output budget, not the shape of the tree. What this run establishes is the price of the shape: seven applications on the critical path instead of three, 130.57 s against the tree's 58.25 s, 2,638 message tokens against 1,483, and $0.0586 against $0.0439, for identical output.

1 What this run is

property	value
run	`real-mb-fidelity__fidelity-chain-f`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`broker` × 8, `worker` × 24
events	93
wall time	130.58 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.96×	total busy time over wall time
parallel efficiency	12.0%	achieved parallelism per rank
idle fraction	88.0%	rank-seconds paid for and unused
peak ranks busy	1	of 8 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	1.35×	slowest rank's busy time over the mean
coordination share	48.1%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	24	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	18.51 / 21.91 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	2,638	0 deferred under rendezvous
tokens in / out	4,398 / 3,027	prompt and completion
cost	$0.0586	0.0194 $/kTok out

3 What the analysis notices

- [warning] Concurrency never exceeded one rank despite a world size of 8: this ran serially.
- [ok] 24 rank reattachment(s) occurred, up to incarnation 9: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	18.04	6.9	1	1	0	1	0	0.0	finalized
1	21.16	18.2	1	1	1	1	607	0.0	finalized
2	17.98	19.3	1	1	1	1	525	0.0	finalized
3	20.15	26.8	1	1	1	1	443	0.0	finalized
4	17.07	31.2	1	1	1	1	361	0.0	finalized
5	13.60	4.9	1	1	1	1	280	0.0	finalized
6	17.59	6.4	1	1	1	1	272	0.0	finalized
7	0.00	0.0	0	0	1	0	150	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

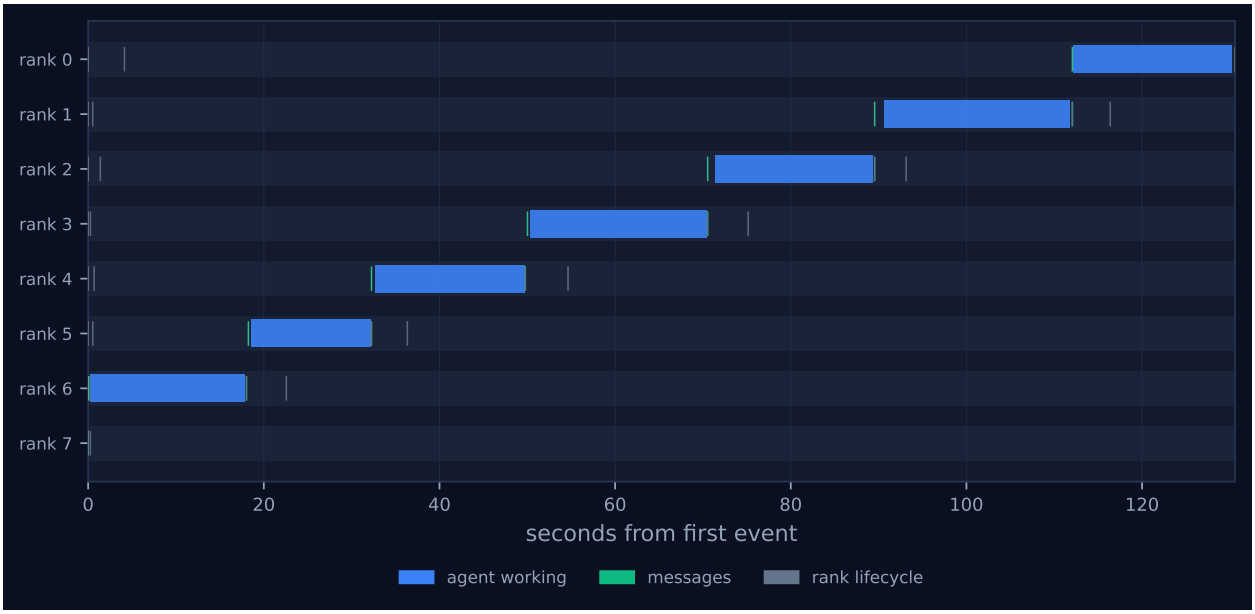

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

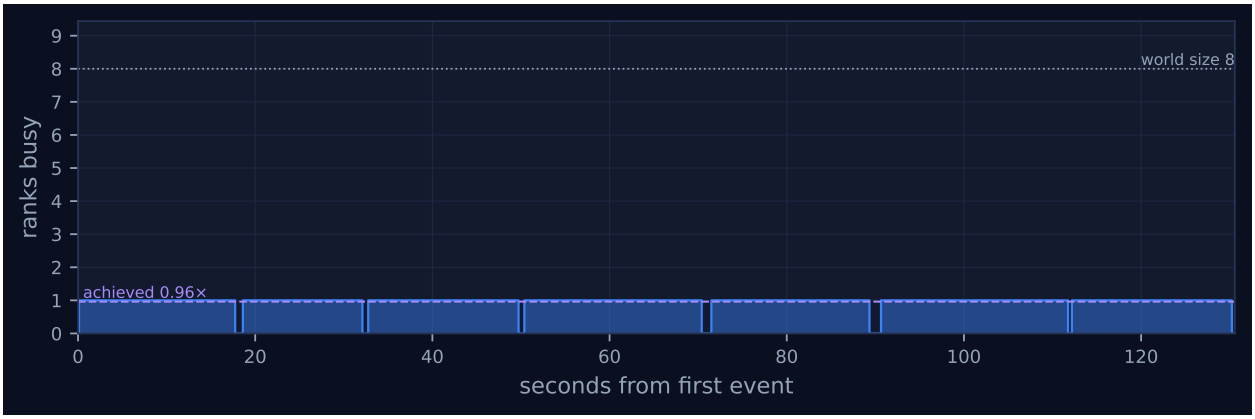

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

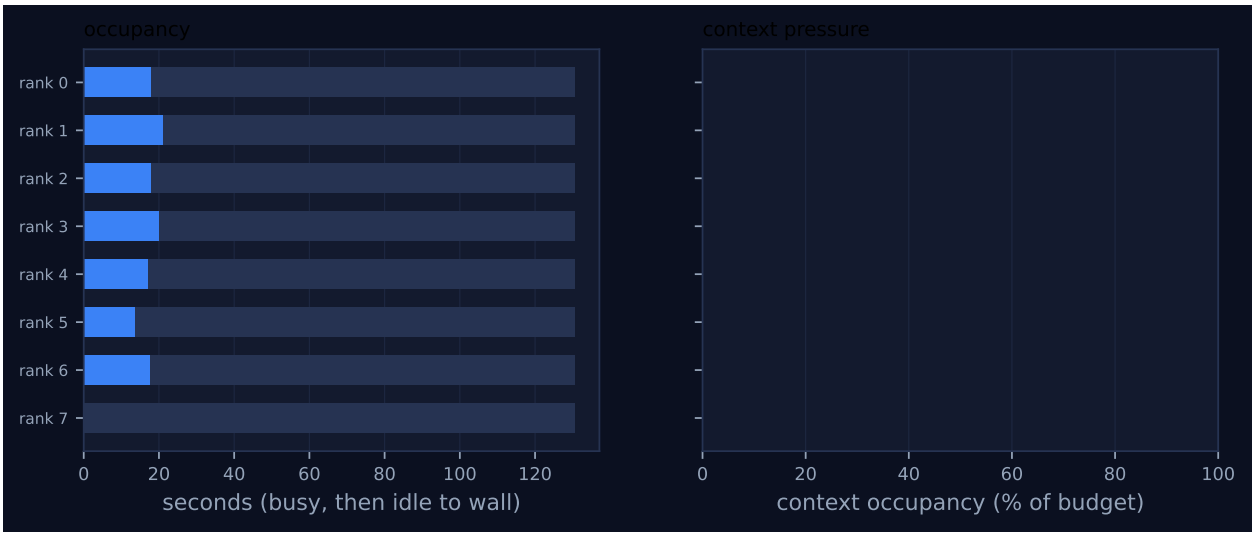

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	chain	–	8	8	7	7	7	7	2,546	130.57	130.57	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is the cleanest picture of serialisation in this archive: a descending staircase of seven bars, exactly one at a time, no overlap anywhere. Rank 6 is busy from 0.02 s to 18.02 s, then rank 5 from 18.25 s, then ranks 4, 3, 2, 1 and finally rank 0, which finishes at 130.58 s. Rank 7 has no bar at all — it sent its leaf report at 0.01 s and left, recording `fold_depth: 0` because it applied the operator zero times. The concurrency plot, Figure 2, is therefore a flat line at one for the whole collective: `max_busy` is 1 of 8 and the serial fraction of active time is 100.0%. There is no straggler to identify and no barrier skew to measure, because the algorithm admits neither; each rank's start is its predecessor's finish, and the 0.13–1.01 s risers between the bars are broker turnaround rather than anything the protocol did.

That structure makes the run a direct measurement of the critical path, and the arithmetic closes. The seven merges took 18.01, 14.01, 17.51, 20.51, 19.01, 22.51 and 18.51 s — median 18.51 s — summing to 130.06 s against a collective span of 130.57 s, so 99.6% of the wall clock is a model generating tokens with nobody else doing anything. Seven times the median predicts 129.6 s. Compare the binomial sibling, where the same seven applications took 120.55 s in aggregate but

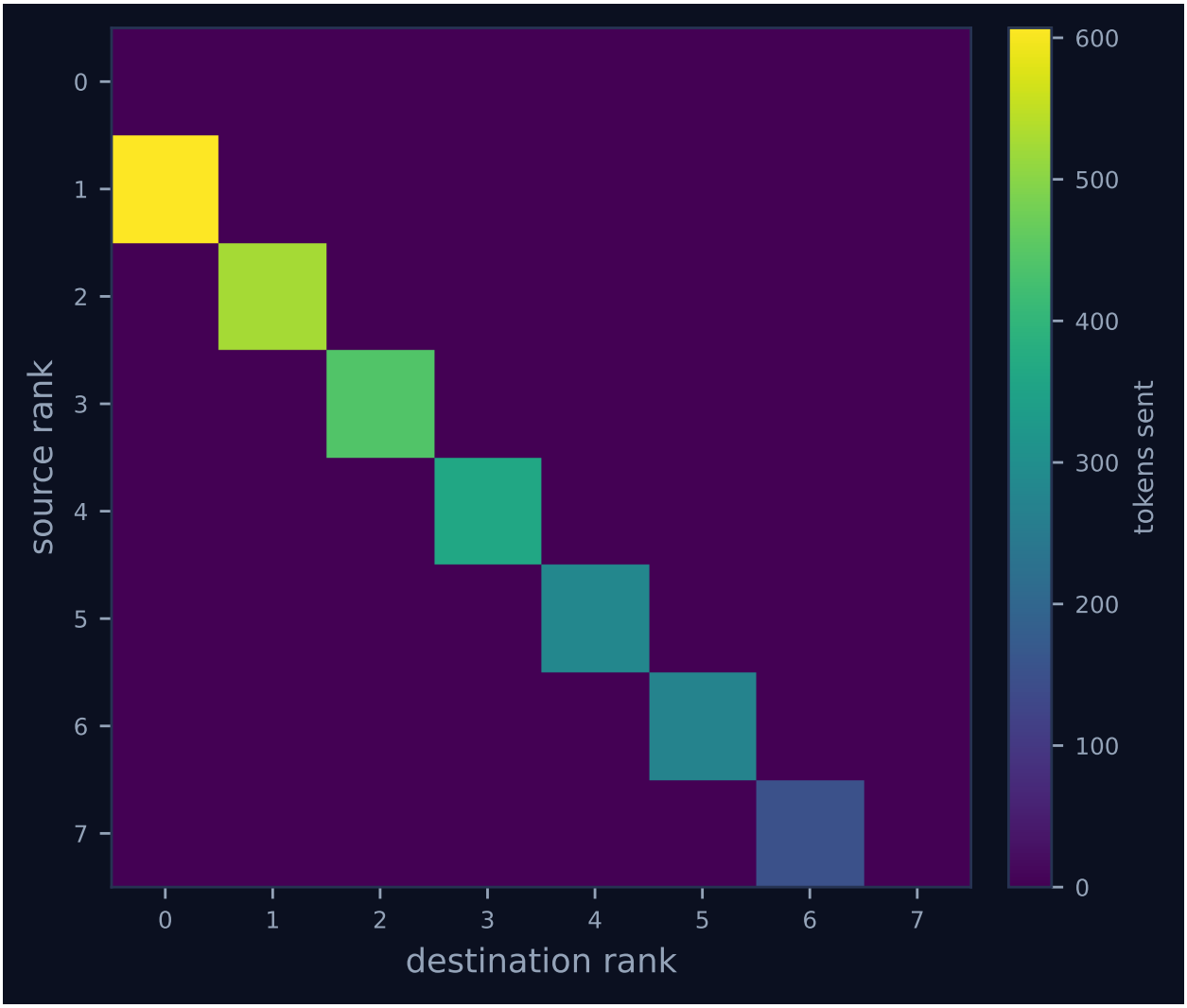

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

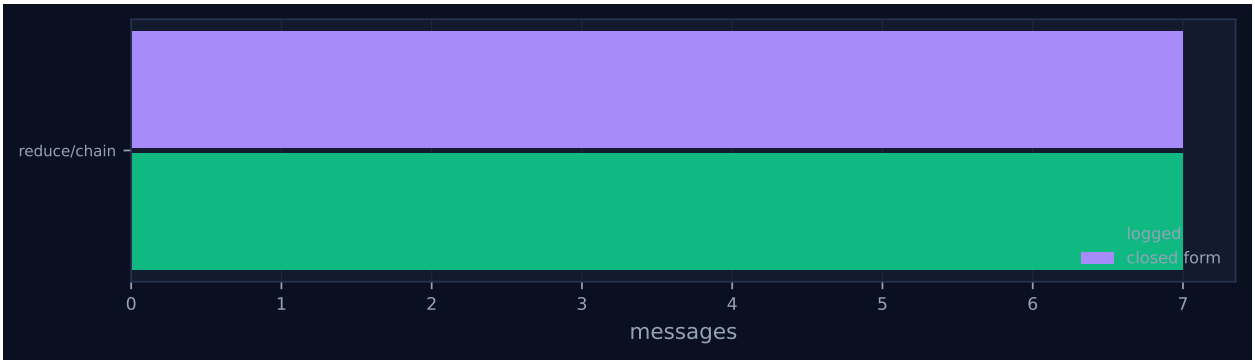

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

only 58.25 s of wall clock because four of them ran concurrently, and the flat sibling, where all seven ran serially at rank 0 for 152.60 s. The chain buys nothing for its depth: it does not reduce messages (7 in all three cases), it does not reduce applications (7 in all three cases), and it is the worst of the three on fabric traffic.

Load imbalance reads 1.18× and here, unusually, that is a real and meaningful number: the seven ranks that merged were busy for 13.60–21.16 s each, which is as even a distribution of work as this operator allows, because every rank performs exactly one application on an accumulator one item larger than the last. It is also the reason the coordination figures are the largest of the three cells. Collective rank-seconds are 502.72 s against a 130.57 s span — 3.85× — because each rank enters the reduce at $t \approx 0$ and blocks in `recv` until its predecessor delivers, so its own `coll.reduce` wall time includes all the waiting. Coordination share is 22.4% and coordination in flight 46.6%, both the highest in the campaign. In the flat sibling the same idleness is invisible to those metrics, because its seven non-root ranks only send and return immediately.

The communication matrix, Figure 4, shows the accumulation as a clean sub-diagonal band from rank r to rank $r-1$ with monotonically growing weight: 150, 272, 280, 361, 443, 525 and 607 tokens. That growth is the mechanism the hypothesis was about, made visible — each hop carries everything gathered so far — and it is why the chain sends 2,638 message tokens where the tree sends 1,483 and the flat reduction 987. It is also why the chain's prompts grow: 442 tokens at the first merge to 838 at the last.

Finally, an artefact large enough to distort the headline table. `job.finish` is emitted at 130.58 s, but the log runs to 280.20 s: the final 10 of 93 events are `rank.init` records for workers re-attaching to rank roles after the job completed, rank 0 reaching incarnation 9. That is 53.4% of the reported wall time, and it is the sparse grey right-hand half of every lane in Figure 1.

7 Interpretation

Most of the run's shape is true by construction. The world size, the four items per rank, the merge prompt, the 900-token nominal budget and the choice of the chain algorithm are harness decisions in `bench_fidelity`; the fold structure is fixed by `algorithms.py`, and both checkable predictions hold — 7 messages against a closed form of 7 and 7 rounds against 7, so `model_checks_agree` is 1 of 1 and Figure 5 shows no red diamond. One thing worth naming, because the hypothesis is usually stated in terms this campaign does not realise, is that the depth contrast here is only two-valued. The `flat` branch of `algorithms.py` sorts contributions by source rank and then applies the *binary* operator $p-1$ times in sequence at the root, so it records `fold_depth = 7` with `rounds = 1`, and its trace shows seven successive `merge:d1`…`merge:d7` calls on rank 0. This run and `-flat-f` are therefore both depth-7 cells with different communication patterns, and the campaign compares 3 against 7 twice rather than 3 against 7 against 1. The single-application-of-p-inputs form is the variadic kernel used only by `kary`, which this campaign did not run; it appears in the `sat-mb-fidelity` and `inc-mb-fidelity-inc` siblings at $k = 4$, depth 2.

The retention result is what emerged, and it refutes the prediction directly, because this is the cell that was supposed to fail. Seven successive model summarisations, each free to compress, cost zero identifiers. Because an aggregate figure can hide a reduction that keeps a complete prefix and erases the tail, I reconstructed the fold rather than trusting the aggregate: the seven accumulators carry identifier sets $\{6,7\}$, $\{5,6,7\}$, $\{4,\ldots,7\}$, $\{3,\ldots,7\}$, $\{2,\ldots,7\}$, $\{1,\ldots,7\}$ and $\{0,\ldots,7\}$ — exactly the chain's fold order from rank 7 down to the root — with 8, 12, 16, 20, 24, 28 and 32 identifiers,

four per rank at every hop. There is no point in the chain at which an earlier contributor thins out. Per-rank retention is genuinely 1.000 for all eight ranks, and `n_hallucinated_ids` is zero, so the operator neither dropped nor invented. This is the strongest single statement in the campaign against the depth hypothesis: if repeated lossy re-summarisation eroded content, the depth-7 chain at $p = 8$ is where it would show, and it does not.

Whole-rank erasure is a real failure mode, but it belongs to the enforced campaign, and what that campaign shows is that depth is irrelevant there too. In `inc-mb-fidelity-inc`, where the payload is high-entropy serials and checksums that cannot be paraphrased shorter and the budget is enforced by contract, retention falls to 0.385 — and it is the *same* 0.385 with the same per-rank pattern at k-ary depth 2, binomial depth 3, and chain and flat depth 7: ranks 0–2 retained completely, rank 3 keeping 1 of 12 items, ranks 4–7 keeping none, rank-position correlation -0.86. Four of eight ranks erased whole, identically under every tree shape. The mechanism is that when every node's output is bound by the same budget B, the last application is the binding one, so the surviving item count is B over the per-item cost regardless of how the items arrived; losses have no slack to compound into. That is a better rule than the one under test, and it makes collective selection a pure cost decision — which is exactly what this run's 130.57 s says about the chain.

The reason nothing was lost in *this* configuration is arithmetic, and quantifying it tells a harness author when to care. Reading accumulator sizes off the flat sibling's left fold, which grows one rank at a time, output tokens track $18.3 + 85.2 \times$ (ranks folded) to within one token over $n = 2 \ldots 7$; at $p = 8$ that extrapolates to 700 tokens and the 900-token budget is first reached at $p \approx 10.3$, about 41 items. This run's own accumulators tell the same story from the other direction: 263, 266, 346, 428, 510, 592 and finally 622 tokens, so even the largest is 69% of the budget. Eight ranks contributing four short items each is below the saturation threshold, and an operator that is never forced to discard is lossless at any depth. At the observed 19.4 tokens per item the budget's capacity is roughly 46 items.

There is a second and stronger reason the retention number cannot bear the weight the hypothesis wanted to put on it: the budget was never enforced. The `agent_calls` row for every one of the seven merges records `contract = {"name": "MergedReport", ..., "max_tokens": null, "semantics": ""}` while `meta` records `{"max_tokens": 900}`. The budget went to the executor as an advisory generation hint, which a worker is free to ignore, and never to the contract checker that would have rejected an overrun and retried with the overrun quantified. All seven calls show `attempt: 1` and the run reports zero contract violations, which under a null bound is uninformative rather than reassuring. For contrast, the `inc-mb-fidelity-inc` fabrics carry `max_tokens: 450` with non-empty semantics, and its `flat` cell contains eight calls rather than seven: `merge:d3` returned 472 tokens on attempt 1, was rejected for overrunning the 450-token bound, and was re-issued as attempt 2, which returned 437 — the enforcement path actually firing. The consequence of leaving it advisory is visible in `sat-mb-fidelity`, still advisory but with a 450-token budget and twelve items per rank: its merges emitted up to 698 tokens, 55% over the stated limit, and still scored retention 1.000. This run did not overrun only because 32 short items fit comfortably under 900. Both facts are needed: the budget was not enforced, and it made no difference here because nothing came close to it.

What the run does establish is the cost of maximum depth at held-constant quality, and it is the campaign's cautionary cell. All three algorithms perform exactly seven operator applications and send exactly seven messages; they differ only in how many applications lie on the critical path and in how much accumulated text each application must be shown. Pooling all 21 merges across the three runs, per-call latency fits $12.31\,\text{s} + \text{output\_tokens}/57.0\,\text{tok/s}$ with $r = 0.789$: about 12 s of

fixed overhead plus generation. The chain pays that overhead seven times in series, which is 2.24× the tree's wall clock, and it also pays the most in tokens — 4,398 prompt and 3,027 completion against the tree's 3,747 and 2,180 — because every hop re-reads and re-emits the whole accumulator. At $0.0586 against $0.0439 that is 33% more money for the same 622-token result. The chain has no compensating advantage of any kind in this configuration, and the one place it might have had one — fidelity, if depth were free of loss in a shallow tree but the chain somehow preserved ordering better — is ruled out by the byte-identical outputs.

One number the metrics do not compute is worth adding, because it is the one axis on which the chain is not the worst. Decomposing each call into `broker.enqueue` → `broker.claim` → `broker.complete`, this run spent only 3.20 s of its 128.79 s of aggregate call latency waiting for a worker to claim a pending call (2.5%), against 12.27 s of 118.67 s in the tree (10.3%) and 26.22 s of 150.93 s in the flat reduction (17.4%). The chain hands each call to a different rank whose worker is already idle and waiting, so attach latency is minimal; the flat root, by contrast, reached incarnation 8 for seven calls and paid roughly 3.7 s of worker attach each time. This is a small effect against a 72 s deficit, but it is the mechanism by which flat manages to be slower than the chain despite needing one communication round rather than seven.

All three flagged findings are correct and none is a defect. The warning that "concurrency never exceeded one rank despite a world size of 8" is the algorithm's definition rather than a fault: a chain reduce serialises by construction, and `max_busy` of 1 with a serial fraction of 100.0% is exactly what a depth-7 fold at $p = 8$ must produce. The 24 reattachments are the broker's normal mode of operation, and the cost-model agreement is the check passing. The real defect in this run's derived artefacts is in a headline figure that the finding list does *not* flag. "Parallel efficiency is 5.6%" and the 94.4% idle fraction both divide by `wall_s`, which `analysis.py` defines as last event minus first event, and 53.4% of that span is post-completion worker churn. Recomputed against the 130.57 s collective span, achieved parallelism is 0.96× rather than 0.45× and the idle fraction 88.0% rather than 94.4%. The distortion matters across the campaign, not just here: the published figures rank the three algorithms 0.31× (binomial), 0.45× (chain), 0.82× (flat), which is the exact reverse of the truth, since binomial is the only cell that achieved any concurrency (peak ranks busy 4, against 1 and 1). Corrected, the ordering is 1.83×, 0.96×, 0.82×. Bounding the analysis window at `job.finish`, or excluding lifecycle-only events from the span, would repair every run whose worker pool outlives its job. That the concurrency warning fires here while the parallel-efficiency note fires on the binomial cell instead is itself a symptom: the binomial cell's efficiency looks worse than this one's only because its post-completion tail is longer.

8 Threats to this reading

The most serious threat is that retention 1.000 may not measure information transport at all. In the compressible configuration `make_fact_report` builds each item as a throughput of $100 + 7 \times \text{rank} + i$ under workload `['nominal', 'degraded', 'saturated'][i % 3]`, so an item's payload is a deterministic function of its identifier, and `fact_retention` scores only whether the identifier appears. A merge that had never seen `F-6-2` could emit it correctly by extrapolating from its neighbours and would be scored as retaining it. This threat is sharpest for exactly this cell, because a depth-7 chain gives the pattern seven opportunities to be re-derived rather than transported, and the metric cannot tell the two apart. Every claim here about what survived is a claim about identifier survival on a guessable payload; the `-inc` variant, with high-entropy serials scored by `value_retention` on the payload as well as the label, exists precisely because of this.

Second, the final result is byte-identical across all three algorithms, and I cannot fully exclude that the worker pool remembered rather than recomputed it. Digest `e1801060bfbe` (1,498 bytes) is the only blob shared by all three runs' content-addressed stores, and three different prompts — 838 tokens here, 832 in the tree, 914 in the flat reduction — produced it. Two of this run's intermediates also coincide with the tree's, its $\{6, 7\}$ merge and its $\{4, 5, 6, 7\}$ merge, the first from an identical prompt and the second from a different one. The runs shared one persistent pool of Cursor subagents and ran sequentially, and rank 0 performed the final merge in all three. The evidence favours convergence, and the discriminating test is the wording register. The population uses a small set of discrete renderings, from the verbatim input at 119 bytes per item down to `[F-0-0] system-0.0: 100 units/s, nominal.` at 46, and a merge inherits its accumulator's register: this run adopts a 68-byte register at its second merge and holds it essentially unchanged through the sixth (68, 67, 67, 66, 66), the flat run holds a different 82-byte register from its first merge to its sixth, and the tree's four subtrees independently adopt four different registers. The only step in any of the three runs where the register is *not* inherited is the last, where 32 items must be rendered and all three paths land on the same minimal encoding — and the title is house style rather than coincidence: all 21 merges open their title with `Consolidated report`, and 20 of the 21 use the exact form `Consolidated report: component groups ...`, the sole exception substituting `from` for the colon. Decisively against wholesale reuse, this run and the flat run share *only* the final result and no intermediate whatsoever: their fold orders are opposite, their accumulators are disjoint, and each run's intermediates track its own fold order and nothing else. The flat run's unique seven-rank accumulator, 2,330 bytes, exists in no other store, and a worker replaying remembered text would not have produced it. Byte-identity of a free-form title across three runs remains a coincidence I can explain but not rule out.

Third, this is one trial of one campaign. The latency ordering against the tree is safe: per-call latency across all 21 merges spans 13.01–27.01 s with a standard deviation of 3.56 s, so the 72 s gap is many times the variability of a single call and is predicted by mechanism — identical application counts, seven on the critical path instead of three. The 22 s gap between this run and the flat reduction is not safe on one trial each; it is consistent with flat's larger root prompts and its sevenfold worker-attach cost, but I cannot separate it from run-to-run variance in model output. The campaign configuration records `reps: 5` and `trials: 32`, neither of which applies to the fidelity bench — those parameters belong to other benches, and reading them as replication here would be wrong.

Fourth, several accounting figures in the generated tables mean less than they appear to. Every `msg.recv` records `admitted: false` because the collective's internal receives pass `admit=False`, so context occupancy is 0% and high water 0 on all eight ranks: this run exercises no context pressure and says nothing about admission, which is notable given that the chain is the one algorithm whose accumulator could plausibly pressure a rank's budget. The `job.finish` payload reports `tokens_deferred: 2638` while `metrics.json` reports 0, because the runtime's counter includes unadmitted receipts and the analysis's counts only rendezvous back-pressure. The reported `rank_position_correlation` of 0.0 is not a measurement of positional fairness but an undefined quantity: at retention 1.000 the per-rank variance is zero, the correlation's denominator vanishes, and the code returns 0.0 by a guard — which matters here because a chain is the shape most likely to show positional bias if any existed. And the $0.0586 is modelled, not billed: 3.0/Mtok in and 15.0/Mtok out from the calibration block reproduce it exactly from token counts estimated with `tokenizer_exact: false`.

Finally, the comparison this run anchors within the paper is not the one the paper quotes. `make_tables.py` populates `\NFidChain` and its companions from whichever broker-executed, non-

capacity-bound block iterates last, which is `sat-mb-fidelity` rather than this campaign, so the published "at equal quality" chain figure is 1,377 s and the speedup 17.8×. This run's figure is 130.57 s and the speedup 2.24×. Both blocks are agent-executed and unsaturated and both support the same direction, but the macro carries no label identifying its configuration, and the two differ by an order of magnitude in effect size — the 1,377 s figure comes from twelve items per rank under a 450-token budget, where each hop's accumulator is three times the size of this one's.